Tokenization

Word · Character · Subword · BPE · WordPiece · SentencePiece

Outline

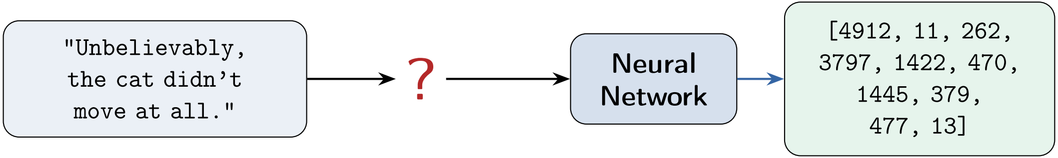
Why tokenize?

Byte-Pair Encoding

Other subword algorithms

Tokenization in practice

Models need numbers, not text



Tokenization is the first step in any NLP pipeline:
split raw text into discrete units (tokens) and map each to an integer ID.
The choice of tokenizer affects **everything** downstream.

Three levels of granularity

Word-level

["Unbelievably", "the", "cat", "didn't", "move"]

Vocab size: ∼100k+

OOV problem

Subword

["Un", "believ", "ably", ",", "the", "cat", "didn", "'t"]

Vocab size: ∼30k–50k

The sweet spot

Character

["U","n","b","e", "l","i","e","v", "a","b","l","y",...]

Vocab size: ∼256 (bytes)

Very long sequences

Used by all modern LLMs

Word-level tokenization and the OOV problem

Vocabulary (fixed at training):

`the, cat, sat, on, mat, dog, run, happy, sad, ...`

Size: 50,000–200,000 words

At test time:

`"The cryptocurrency market plummeted after the CEO's tweet."`

Words not in vocab → `[UNK]` [UNK] market [UNK] after the [UNK] tweet.

Problems: • New words, names, typos, or **casing** ("The" vs "the") → `[UNK]`
• Huge vocabulary → huge embedding matrix • Morphology lost: "run", "runs", "running" are unrelated tokens

Character-level: no unknowns, but. . .

Input: "The cat sat on the mat."

`The | cat | sat | on | the | mat | .`

7 tokens (word-level)

`T|h|e| |c|a|t| |s|a|t| |o|n| |t|h|e| |m|a|t|.`

23 tokens (character-level)

Drawbacks:

- Sequences ∼3–5× longer (here $23/7 \approx 3.3\times$)
- $O(n^2)$ attention cost explodes
- Characters carry little meaning
- Harder long-range dependencies

Advantages:

- Tiny vocab (∼256 *byte* values)
- Zero unknown tokens
- Works for any language
- Handles typos, code, URLs

Here "character" means a **byte** (∼256 values); full Unicode has >100k distinct symbols, which is why models go byte-level (next). Too fine-grained alone — but it inspires **subword** methods.

Subword tokenization: the key insight

Common words stay whole: `the`, `cat`, `and`

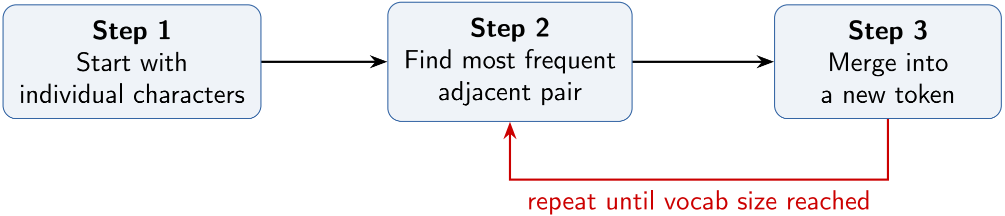
Rare words are split into known pieces: `un` + `believ` + `ably`

"playing"	$\longrightarrow$	`play` + `ing`
"unhappiness"	$\longrightarrow$	`un` + `happi` + `ness`
"ChatGPT"	$\longrightarrow$	`Chat` + `G` + `PT`
"brrrr"	$\longrightarrow$	`br` + `rr` + `r`

Vocab size ∼30k–50k • No `[UNK]` tokens • Reasonable sequence lengths • Morphology is partially captured

Byte-Pair Encoding (BPE): the idea

Training: learn merge rules from a corpus. **Inference:** apply merge rules to new text.



Originally a **data compression** algorithm (Gage, 1994). Adopted for NLP by Sennrich et al. (2016). Used by **GPT**, **GPT-2**, **RoBERTa**, **BART**, **LLaMA**.

BPE: worked example

Training corpus (word × frequency): cheese ×10, cheek ×8, cheap ×5

Initial vocab: c, h, e, s, k, a, p
Splits: c h e e s e (×10) c h e e k (×8) c h e a p (×5)

↓

Merge 1: most frequent pair = (c, h) freq = 10+8+5 = 23
Splits: ch e e s e (×10) ch e e k (×8) ch e a p (×5)

↓

Merge 2: most frequent pair = (ch, e) freq = 10+8+5 = 23
Splits: che e s e (×10) che e k (×8) che a p (×5)

↓

Merge 3: most frequent pair = (che, e) freq = 10+8 = 18
Splits: chee s e (×10) chee k (×8) che a p (×5)

"cheap" stops merging at step 3 (no "ee"). Note: at step 1, (c,h) and (h,e) are both freq 23 — BPE breaks such ties by a fixed order (here, the first pair seen).

Merge rules (ordered):

1. c + h → ch
2. ch + e → che
3. che + e → chee

⋮

Vocab after 3 merges:
c, h, e, s, k, a, p,
ch, che, chee

BPE: tokenizing new text

Given the learned merge rules, tokenize a word **never seen** in training:

Tokenize "cheeks":

Start: `c h e e k s`

Rule 1 (`c+h`): `ch e e k s`

Rule 2 (`ch+e`): `che e k s`

Rule 3 (`che+e`): `chee k s`

Key point:

Apply merge rules in the *same order* they were learned.

"cheeks" was never in the training corpus, but BPE splits it into known pieces: `chee` + `k` + `s`

Each ID = the token's index in the vocab.

Result: `["chee", "k", "s"]` → token IDs `[9, 4, 3]`

Byte-level BPE (GPT-2, GPT-3, LLaMA)

Standard BPE

Base vocab = Unicode characters

Vocab size = ∼30k–50k

Unknown chars → `[UNK]`

Problem: Chinese, emoji, etc. can hit unknown characters

upgrade →

Byte-level BPE

Base vocab = **256 byte values**

Vocab size = 256 + merges (GPT-2: 256 bytes + 50,000 merges + 1 special = 50,257)

No `[UNK]` ever

Any byte sequence is representable

Every text is ultimately a sequence of bytes (UTF-8 encoding).
By starting from **bytes** instead of characters, BPE can tokenize *any* input: English, Chinese, Arabic, code, emoji, binary data — all with the same vocabulary.

WordPiece (BERT, DistilBERT)

BPE

Merge criterion:
most frequent pair

Greedy count-based

Used by: GPT, LLaMA, etc.

WordPiece

Merge criterion:
pair that **maximizes likelihood** of the training corpus

Used by: BERT, DistilBERT

$$\text{score}(a, b) = \frac{\text{freq}(ab)}{\text{freq}(a) \times \text{freq}(b)}$$

$\approx$ PMI: rewards *surprising* co-occurrence

Notation: WordPiece marks *continuation* subwords with ##

"unbelievably" $\rightarrow$ `["un", "##bel", "##ie", "##va", "##bly"]` (the real BERT split)

BPE instead marks *word-initial* subwords (e.g., GPT-2 uses Ġ = space prefix)

WordPiece: BPE vs WordPiece merge decision

Corpus (all split to characters): cheese (×10), cheek (×8), cheap (×5), seep (×7).
Which pair should we merge: $(\mathtt{e},\mathtt{e})$ or $(\mathtt{e},\mathtt{p})$?

BPE: raw pair count

$(\mathtt{e},\mathtt{e})$: cheese 10 + cheek 8 + seep 7 = 25

$(\mathtt{e},\mathtt{p})$: seep 7 = 7

("cheap" is c-h-e-a-p: no adjacent e, p)

Winner: $(\mathtt{e},\mathtt{e})$ — more frequent

WordPiece: likelihood score

score $= \frac{\mathrm{freq}(ab)}{\mathrm{freq}(a)\,\mathrm{freq}(b)}$, freq(e)= 65, freq(p)= 12

$(\mathtt{e},\mathtt{e})$: $\frac{25}{65\times 65} = 0.0059$

$(\mathtt{e},\mathtt{p})$: $\frac{7}{65\times 12} = 0.0090$

Winner: $(\mathtt{e},\mathtt{p})$ — higher score

WordPiece favors pairs whose co-occurrence is **surprising** given how common each piece is.
"e" is everywhere, so "ee" is expected; "ep" co-occurring is more informative.

Unigram LM and SentencePiece

Unigram LM

Start with a *large* vocab

Iteratively **remove** tokens that hurt likelihood the least

Top-down (vs BPE's bottom-up)

Used by: T5, ALBERT, XLNet

Kudo, 2018

SentencePiece

Not an algorithm — a **library**

Treats input as raw byte stream (no pre-tokenization needed)

Supports both **BPE** and **Unigram**

Language-agnostic: no need for space-based word splitting

Kudo & Richardson, 2018

BPE = bottom-up (merge frequent pairs) vs **Unigram** = top-down (prune unlikely tokens)

Unigram gives each token a probability and segments a word into the piece-sequence with the highest product of probabilities; training drops the pieces whose removal costs the least likelihood.

Both converge to similar subword vocabularies in practice.

Why tokenization matters: LLM quirks

Bad at arithmetic

"`12345`" → `["123", "45"]`
The model never sees the individual digits together!

Poor non-English efficiency

English "hello" = 1 token
Korean "`annyeong`" = 3–5 tokens
Same meaning, 3–5× the cost!

Sensitive to formatting

"`Hello World`" and
"`Hello  World`" produce
different token sequences.

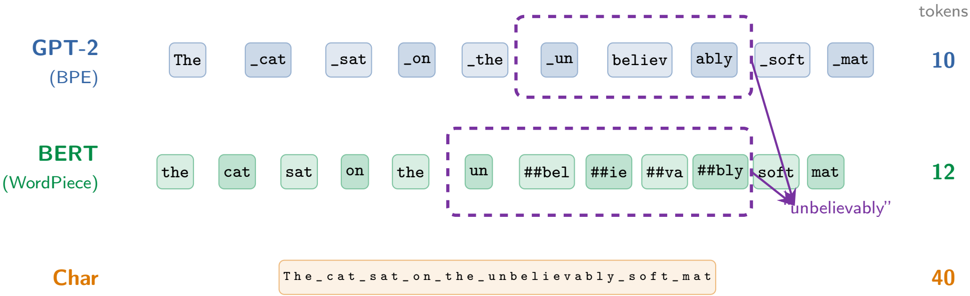
Can't count letters

"How many r's in strawberry?"
"`straw`" + "`berry`" — the model can't see individual letters.

Many apparent "reasoning failures" of LLMs are actually **tokenization artifacts**.
The model literally cannot see what you think it sees.

Visualizing: the same sentence, different tokenizers

Input: "The cat sat on the unbelievably soft mat"



Same input, very different representations. "unbelievably" = **3 tokens** (GPT-2), **5 tokens** (BERT), **12 characters**.

GPT-2's _ marks a leading space; BERT's ## marks a word continuation.

Fewer tokens = cheaper inference (less compute per sequence).

Special tokens

[CLS]
Classification token (BERT)

[SEP]
Separator between segments

[PAD]
Padding to equal length

[UNK]
Unknown token (fallback)

$\langle$**BOS**$\rangle$
Beginning of sequence

$\langle$**EOS**$\rangle$
End of sequence

[MASK]
Masked position (BERT MLM)

Special tokens are **not** in the original text — they're added by the tokenizer to give the model structural signals: where sequences begin/end, what to predict, etc.

Comparison of tokenization methods

Method	Direction	Criterion	Vocab size	Used by
BPE	Bottom-up	Frequency	30k–50k	GPT, LLaMA
WordPiece	Bottom-up	Likelihood	30k	BERT
Unigram	Top-down	Likelihood	30k–50k	T5, XLNet
Byte BPE	Bottom-up	Frequency	50k–100k	GPT-2/3/4
Character	—	—	256	ByT5

In practice, the differences between BPE, WordPiece, and Unigram are **small**. What matters most: vocab size, training corpus, and whether byte-level is used. **Byte-level BPE** is the current default for new large language models.

Practical: tokenizers in action

```python
# GPT-4 tokenizer (tiktoken:
# OpenAI's fast BPE impl in Rust)
import tiktoken
enc = tiktoken.encoding_for_model(
  "gpt-4")
tokens = enc.encode(
  "Hello world!")
# [9906, 1917, 0]
enc.decode(tokens)
# "Hello world!"
```

```python
# BERT tokenizer
from transformers import
  AutoTokenizer
tok = AutoTokenizer.from_pretrained(
  "bert-base-uncased")
tok.tokenize(
  "unbelievably")
# ["un", "##bel", "##ie",
#  "##va", "##bly"]
```

Try it yourself: `https://platform.openai.com/tokenizer`

Paste any text and see how GPT tokenizes it. Pay attention to: numbers, non-English text, code, and whitespace.

Trend: byte-level tokenizers

Why go byte-level?

- **No unknown tokens** — ever
- Truly **language-agnostic**: same vocab for all scripts
- Handles code, URLs, binary
- No brittle *pre-tokenization* (the regex word-split done first)
- Simpler, more robust pipeline

Models & approaches

- **ByT5** (2022): pure byte-level, no merges at all
- **MegaByte** (2023): byte-level with patching for efficiency
- **GPT-4**, **LLaMA 3**: byte-level BPE with large merge tables
- Trend toward *fewer assumptions* about input structure

The trade-off: pure byte-level models produce **longer sequences** ($\sim 4\times$), increasing compute cost. Byte-level *BPE* (with merges) is the current sweet spot: byte-safe base vocabulary + subword compression.

Further reading

Subword Tokenization

- Sennrich et al. (2016), "Neural Machine Translation of Rare Words with Subword Units" (BPE)
- Kudo & Richardson (2018), "SentencePiece: A simple and language independent subword tokenizer"
- Kudo (2018), "Subword Regularization: Improving NMT Models with Multiple Subword Candidates" (Unigram LM)

WordPiece & Byte-Level

- Schuster & Nakajima (2012), "Japanese and Korean Voice Search" (WordPiece)
- Wang et al. (2020), "Neural Machine Translation with Byte-Level Subwords"

Practical Resources

- HuggingFace Tokenizers library documentation — `huggingface.co/docs/tokenizers`
- OpenAI Tiktoken (fast BPE in Rust) — `github.com/openai/tiktoken`
- OpenAI Tokenizer tool — `platform.openai.com/tokenizer`

Questions?

Next: Evaluation — Perplexity, BLEU, ROUGE